What Does a Scheduler Lose Without an Exact Memory Footprint? Existing Estimators and Provisioning Policies on Disaggregated Memory

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

A scheduler for a cluster with disaggregated memory can pack jobs only as tightly as its knowledge of each job's memory footprint allows. We measure how much an exact footprint is worth to such a scheduler, and how much of that worth the existing sources of footprint knowledge recover. We trace 25 programs at three inputs each, HPC kernels, interpreters, a compiler and two LLM inference engines among them, with a full instrumentation trace as the truth and with Massif, DHAT, heaptrack, the gperftools heap profiler and sampled RSS, and we replay the measured footprints through DMSched-Sim, a simulator of a cluster with local DRAM and a shared memory pool in which only the footprint oracle changes. We also evaluate three published provisioning policies, Pond, the dynamic provisioning of Zacarias et al. and KS+, and replay jobs of the SURF Lisa trace with their measured memory. At 80% offered load the exact footprint curve gives a mean response time 26.4× shorter than the user's request and 5.3× shorter than the exact peak, and it needs 5.0 TB of memory for a service level that the exact peak meets with 6.0 TB and the request with 11.0 TB. No existing source recovers this gain. The heap profilers report the right peak for most programs but read a compiler and an LLM engine at 0.14 to 0.17 of their peaks, because they see only the heap, so no margin makes them safe without over-reserving every other job. Pond holds the whole request and is slower than the request itself. Dynamic provisioning comes close to the exact peak only by reading the coming minutes of each job's use from its trace; a version that looks back, as a running system must, falls behind the exact peak. KS+'s regressions on input size fail when the job's input is far from the ones it has seen. On the Lisa trace the exact curve completes 18 to 31% more jobs than the exact peak once memory is scarce. The exact curve itself costs 227 times the program's run to obtain. We derive from these results the requirements a footprint estimator must meet to be useful to a scheduler.

1 Introduction

A batch scheduler places a job only after it decides how much memory to reserve for it, and on a cluster with disaggregated memory that decision also sets how much of the shared pool the job takes from other jobs. Disaggregated memory attaches a pool of memory, over CXL or RDMA, that every node can use in addition to its local DRAM [11, 23]. The pool lets a cluster provision less local memory per node, but only if the scheduler can pack jobs tightly, and it can pack them only as tightly as its knowledge of each job's memory allows. That knowledge is weak today. Peng et al. found that over 90% of LLNL jobs use under 15% of a node's memory [15], and Zacarias et al. showed that a uniform 60% overestimate of memory demand cuts the throughput of a disaggregated system by about 25% and raises the mean response time about five times [24].

Two kinds of knowledge are possible. A *peak* is one number per job, held for the whole run. A *footprint curve* gives the memory the job needs at every point of its run, so a scheduler can let two jobs whose peaks do not overlap share the same memory. The usual sources of either are the user's request, a heap profiler such as Massif [18], DHAT [20], heaptrack [21] or the gperftools heap profiler [9], the resident set size (RSS) of an earlier run, and three kinds of published policy: a learned predictor of a job's untouched memory (Pond [11]), a monitor that resizes a running job's allocation (dynamic provisioning [25]), and a model of a task's memory over time learned from its earlier executions (KS+ [2]).

However, it is unclear how much a scheduler loses with these sources against an exact footprint, and why. The simulators that study scheduling on disaggregated memory either model one run in cycle detail with no scheduler [17, 4, 5, 22], or treat a job's memory as one declared peak with a synthetic error [24, 23]. None feeds the scheduler the footprints that real profilers report, and none compares them with an exact footprint curve. Therefore, we measure every source of footprint knowledge listed above on the same programs and replay the results through one scheduler, with everything but the footprint oracle held fixed.

In this work, we make the following contributions:

- We build a corpus of footprint curves for 25 programs at three inputs each, with a full instrumentation trace as the truth, the curves and peaks of four heap profilers and sampled RSS, and what each method costs (Section 4).
- We build DMSched-Sim, a cluster simulator with local memory and a shared pool, an EASY-backfilling scheduler that admits a job only if its whole reservation curve fits, a remote-memory slowdown model, OOM kills, and implementations of Pond, dynamic provisioning and KS+ checked against their authors' code where it is published (Section 3).
- We show how much of an exact footprint's gain comes from knowing the curve rather than the peak, how much each existing source recovers, why each falls short, and how much memory each needs for the same service (Section 5), on a synthetic stream, on a stream of LLM jobs and on the SURF Lisa trace.
- We derive the requirements a footprint estimator must meet to be useful to a scheduler (Section 7).

2 Background and related work

Disaggregated memory simulators. DRackSim [17], CXL-ClusterSim [4], CXL-DMSim [5] and CXLMemSim [22] model the timing of accesses to pooled memory for one run, in cycle detail or from a trace, and none of them has a job scheduler. The BSC Slurm simulator extended by Zacarias et al. schedules real job traces on a disaggregated system [23], but it represents a job's memory as one peak, and its study of accuracy multiplies that peak by a synthetic overestimate [24]. Boezennec et al. study the allocation of disaggregated memory to jobs with piecewise-constant memory phases [3], and CFM schedules jobs against per-job slowdown curves measured offline [1].

Provisioning policies. Pond predicts the untouched memory of virtual machines with a gradient-boosted model from their customers' history, places that memory in a CXL pool, and moves a virtual machine to local memory when it touches more than predicted [11]. The dynamic provisioning of Zacarias et al. resizes each job's allocation every five minutes to its use, read from usage traces, and fails and restarts a job whose growth does not fit [25]. KS+ learns a workflow task's memory

over time as a step function whose segments it regresses on the input size of earlier executions, and reduces wasted GB-seconds by 51% against peak-only prediction [2]. Pond and dynamic provisioning act on a job's allocation while it runs or from its owner's history, and KS+ needs earlier executions of the same task.

Footprint estimators. Massif samples the heap at up to 100 snapshots by default, on a time axis of instructions or of milliseconds under Valgrind [18], and with `-pages-as-heap` it counts mapped pages instead of heap blocks. DHAT reports the heap at its global peak (t-gmax) and at exit, and the time of the peak, but no time series [20]. heaptrack records every allocation and can export a heap curve [21], and the gperftools heap profiler writes a dump each time the heap grows by a set amount [9]. All four count requested heap bytes, so they miss memory a program maps itself and count blocks that are allocated but never touched.

3 Simulator

DMSched-Sim replays one stream of jobs once per footprint oracle, so that any difference between two runs comes from what the scheduler knows about memory. It is a time-slotted simulator (one slot is one minute) written in Python.

System model. The cluster has 32 nodes, each with 32 cores and 128 GB of local DRAM, and one shared pool of 1 TB (one sixth of all memory). A job's memory is placed in its node's free local memory first, and the rest goes to the pool. A job on several nodes holds its footprint on each of them, its nodes share the pool, and it runs as long as its slowest node.

Jobs. A job has an arrival time, a core count, a base runtime (its runtime with all memory local), a true footprint curve $a(p)$ over its progress $p \in [0, 1]$, and a reservation curve $r(p)$ set by the oracle. Curves are defined over progress rather than time, so a job slowed by remote memory stretches its curve in wall-clock time.

Scheduler. The scheduler is first come, first served with EASY backfilling [12] over a window of 32 queued jobs. Each node keeps a timeline of committed local memory and cores, and the pool keeps one of committed memory. A job starts on a node only if, at every slot from now to its projected end, the part of its reservation that does not fit in the node's free local memory fits in the free pool. With a time-varying reservation, two jobs whose peaks do not overlap can therefore share the same memory. When the head of the queue cannot start, the scheduler reserves its earliest feasible start, and a later job may start early only if it fits beside that reservation. Among the nodes that fit, the scheduler picks the one with the least pool use and then the least free local memory left over.

Remote-memory slowdown. Following the linear models of CFM [1] and Boezennec et al. [3], a job's runtime is stretched by the share of its resident GB-minutes that lie above its local reservation:

$$T = T_{\text{base}}\,(1 + \beta f_{\text{remote}}), \qquad f_{\text{remote}} = \frac{\sum_t \max\bigl(0,\, a(t) - \ell(t)\bigr)}{\sum_t a(t)}, \tag{1}$$

where $\ell(t)$ is the local part of the reservation and $\beta = 0.5$, so a job whose memory is entirely remote runs 1.5 times as long. The runtime and the placement are solved together by fixed-point iteration when the job is admitted.

OOM kills. If the true footprint exceeds the reservation at any slot, the job is killed there, its work is lost, and it is requeued at the head of the queue with its reservation multiplied by 1.5, as a batch system that raises a job's memory limit after an out-of-memory failure would do. The policies that resize a running job handle growth as Section 3.3 describes.

Metrics. We report the mean response time (completion minus arrival), the mean bounded slowdown (response over $\max(T_{\text{base}}, 10\,\text{min})$), the number of OOM kills, and the reserved and the resident memory as shares of all memory, over the jobs that arrive between the 10th and the 90th percentile of arrival times.

Validation. Unit tests check that two jobs with non-overlapping peaks share a node under their curves but run one after the other under their peaks, that an under-reservation is killed where the footprint passes it, that backfilling never delays the head of the queue, that a fully remote job runs $1+\beta$ times as long, and the behaviour of each published policy. The published policies are checked against their authors' code where it is published (Section 3.3), and Appendix A reproduces the authors' own results.

3.1 Job streams

The main experiments use a controlled stream of 3,000 jobs in which memory is the contended resource. Each job has 1 to 16 cores, a lognormal base runtime with a median of 30 minutes clipped to 5 to 720 minutes, and a lognormal memory request with a median of 100 GB. What a job really uses is its request multiplied by a ratio $u \leq 1$ taken from the LANL CM5 log of the Parallel Workloads Archive [6], the archive log that records both requested and used memory (median $u = 0.79$ over its 94,335 jobs with both fields). The shape of the job's footprint, and what every estimator reports for it, come from one corpus program at its largest input (Section 4), chosen at random, scaled so that the truth's peak is the job's peak and every estimator keeps its own over- or under-estimate. Arrivals are spread over a span set so that the *offered load*, the resident GB-minutes arriving per minute over the cluster's total memory, takes the values 0.4 to 1.2. Each point is three seeds.

A real trace with measured memory. The SURF Lisa trace [19] joins 1.6 million Slurm jobs of a 343-node cluster with their nodes' Prometheus metrics, sampled every 30 s from June to November 2022. A node's memory is a job's only while the job has the node to itself, so we keep a job only if no other job shared any of its nodes at any sample and at least 80% of the samples its runtime implies are present: 88,767 jobs of at least five minutes, 1,429 of them on 2 to 22 nodes. A job's memory is its nodes' active memory above each node's idle level. We replay windows of 3,000 consecutive jobs in two ways. In the *measured* stream every job keeps its own curve, and only the oracles that need nothing but the job's own memory apply. In the *hybrid* stream every job keeps the trace's arrival, runtime, node count and measured peak, with a corpus program's footprint shape scaled to that peak, so that every estimator applies. Lisa records no memory request, so a job requests Slurm's default on the simulated cluster, the node's memory per core times the cores it uses, and at least its peak. Both streams run on a cluster of 32 nodes with 16 cores each, as Lisa's, a pool of a quarter of the local memory, and local memory per node swept from Lisa's 103 GB down to 4 GB; the offered load is set against each cluster's own memory.

3.2 Two truths

A footprint can be counted as the bytes a program touches or as the pages a kernel holds for it, and a kernel charges and OOM-kills a process for the latter. The two differ by page granularity, by memory an allocator keeps after a free, and by the runtime's own pages. We run every experiment with two truths. With *bytes touched* as the truth, the scheduler must cover the footprint a page-perfect system would need. With *RSS* as the truth, it must cover the resident set sampled every 5 ms.

3.3 Oracles and margins

An oracle turns what is known about a job into its reservation curve. Curve oracles follow a measured curve, coarsened to 20 windows over progress, each holding its largest value and widened to its neighbours, because each method's clock (instructions or milliseconds under Valgrind for Massif, seconds for the RSS) maps to wall-clock progress only approximately. Peak oracles hold one value for the whole run. The *exact curve* and the *exact peak* are the truth's own curve and peak, the bounds no estimator can pass. The estimators are Massif's curve (heap, or mapped pages with `-pages-as-heap`) on its instruction clock and on its millisecond clock, heaptrack's curve, DHAT's peak and two curves built from DHAT's three recorded points (Section 6), gperftools' peak, the RSS peak of an earlier run of the same input, the user's request, and an ML peak, the request times a 0.95-quantile gradient-boosted prediction of u from the request, core count, requested time and the user's and executable's history, trained on half of the CM5 users, as Pond's model predicts untouched memory [11].

The policies that resize a running job are three. *Reactive resizing* holds the request for five minutes, then each five minutes the past five minutes' largest use plus 10% headroom, and grows the job from free memory when its use exceeds its allocation. *Dynamic provisioning* [25] starts the job with its request and every five minutes resizes its allocation to the largest use of the coming five minutes, as the authors' simulator reads it from the usage trace, local memory first and then the pool; a resize is committed only when it happens, and a resize that does not fit fails the job, which restarts from the beginning. A running system cannot read the coming interval, so its *look-back* variant resizes to the past five minutes' largest use, grows the job on demand a minute at a time, and runs a job with its request held after three restarts, as the authors suggest for jobs that fail repeatedly. The *Pond* oracle [11] holds the job's whole request but keeps only the request minus its predicted untouched memory in local DRAM, the rest in the pool, with the untouched share predicted at the 0.98 quantile (Pond's target of 98% of jobs within their margin) and a QoS monitor that moves the pool part to local memory five minutes after the job first touches it. The *KS+* oracle [2] learns four segments of a non-decreasing step function from the RSS of the program's other corpus inputs, regresses each segment's end and peak on the input size, ends segments 15% earlier and raises peaks 10%, and after an OOM kill pulls the later segments forward to the failure or raises the last peak 20%.

We checked these policies against their authors' published code. Our KS+ matches the authors' segmentation, prediction and retry on random inputs. The dynamic-provisioning code of the authors' Slurm simulator matches ours in admission against current allocations, remote-first release, local-first growth and restart with the original request; it also recomputes the slowdown at every resize and charges 10 s per growing resize, which we leave out in the policy's favour. Pond's artifact holds none of its policies, so our Pond follows its paper.

Every estimate needs a margin, because an estimate below the truth causes OOM kills. A curve oracle's margin is headroom added as a share of its own peak, and a peak oracle's margin multiplies

Table 1: The corpus at each program's largest input: the true peak (bytes touched), the mean over the run as a share of the peak, each method's peak as a share of the true peak, and the time each method takes over the native run.

		Footprint (ratios to the true peak)							Cost (time / native)			
Program	Input	Peak (MB)	Mean/peak	RSS	Massif	DHAT	heaptrack	Native (s)	Massif	DHAT	heaptrack	Full trace
2mm	LARGE	37.1	0.96	1.03	1.00	1.00	1.00	8.6	5.6	15.2	2.3	357
atax	LARGE	32.1	0.87	1.04	1.00	1.00	1.00	0.1	5.7	9.8	2.5	122
GCC cc1	large	105.5	0.69	1.27	0.16	0.15	0.15	1.8	11.0	68.1	2.2	206
darknet	alexnet	269.9	0.86	1.07	1.97	1.97	1.97	3.0	10.5	45.9	1.7	773
GAP bfs	18	72.8	0.36	1.05	0.99	0.99	0.99	1.5	3.0	5.7	1.6	83
GAP cc	18	72.0	0.36	1.06	1.01	1.01	1.01	2.2	2.2	3.9	1.1	56
GAP pr	18	72.4	0.40	1.05	1.00	1.00	1.00	2.8	2.3	5.3	1.2	123
GAP sssp	18	135.8	0.36	0.95	1.00	1.00	1.00	2.5	2.4	4.4	1.1	116
gemm	LARGE	29.1	1.00	1.04	1.00	1.00	1.00	6.1	5.7	15.2	1.1	356
HPCCG	64	102.7	0.99	1.03	1.02	1.02	1.02	1.7	10.5	24.8	1.4	310
jacobi-2d	LARGE	27.1	1.00	1.04	1.00	1.00	1.00	12.8	7.3	16.2	1.5	275
Kripke	16	274.6	0.88	1.02	1.00	1.00	1.00	3.4	4.8	50.7	1.4	473
llama2.c	110M	449.7	0.98	1.14	0.17	0.17	0.17	14.9	3.6	20.5	1.3	41
llama.cpp	135m-n128	352.9	0.89	1.08	1.05	1.05	1.05	1.5	137.1	279.6	2.4	604
Lua	1	57.1	0.53	1.15	1.02	0.92	0.92	4.2	4.3	28.0	1.8	161
LULESH	60	151.1	0.62	1.02	1.00	1.00	1.00	5.5	7.9	32.2	1.3	458
miniVite	32768	15.4	0.51	1.99	1.55	1.53	1.42	1.8	22.9	36.9	4.5	322
MiniFE	80	256.5	0.98	1.04	1.03	1.02	1.02	9.2	6.7	23.8	1.1	273
NPB CG	A	27.4	1.00	1.10	0.99	0.99	0.99	1.9	7.8	15.5	1.2	237
NPB IS	A	69.7	0.81	1.03	0.99	0.99	0.99	2.4	2.7	7.0	1.0	61
NPB MG	W	58.5	0.98	1.04	0.99	0.99	0.99	0.5	4.0	8.8	1.3	107
Perl	1	56.1	0.67	1.15	1.04	0.97	0.96	1.8	5.2	44.8	1.9	145
Python	1	95.9	0.37	1.19	1.29	1.01	0.14	5.6	12.5	175.5	1.0	272
Quicksilver	100000	42.2	1.00	1.28	1.02	1.01	1.01	5.5	6.3	44.5	1.2	227
SQLite	1	77.8	0.43	1.06	1.01	1.01	1.01	1.2	7.1	39.0	1.6	204

its peak. Besides fixed margins of 10% and 30%, we use *calibrated* margins that a site could set from its own history. For each oracle and program we compute the smallest margin with which the oracle never falls below the truth, and a job receives the 90th percentile (*calibrated 90%*) or the largest value (*calibrated 100%*) of the other programs' margins, so no oracle is tuned on the program it predicts. The request, ML and reactive oracles have no profile to calibrate and keep a 10% margin, and the dynamic-provisioning, Pond and KS+ oracles use none.

4 Footprint corpus

We ran 25 programs at three inputs each on a 32-core node with 62 GB of memory (Table 1): four PolyBench kernels [16], miniVite [8], the GAP benchmarks bfs, pr, cc and sssp, Lua, Python, Perl and SQLite workloads, GCC's cc1 compiling three source files, HPCCG, LULESH, NPB IS, CG and MG, MiniFE, Quicksilver, Kripke, darknet classifying images with AlexNet, and two LLM inference engines, llama2.c [10] with three models and llama.cpp [7] with SmolLM2-135M and 360M. Both engines map their weights from the model file rather than allocating them. The native runs take 0.12 to 14.9 s, and the true peaks at the largest inputs are 15 to 450 MB. Every (program, input) ran alone with its RSS sampled every 5 ms; under a full instrumentation trace with Pin [13], which records the bytes touched over time and is the truth; under Massif and Massif with `-pages-as-heap`, each with its instruction clock and with milliseconds; and under DHAT, heaptrack and the gperftools heap profiler.

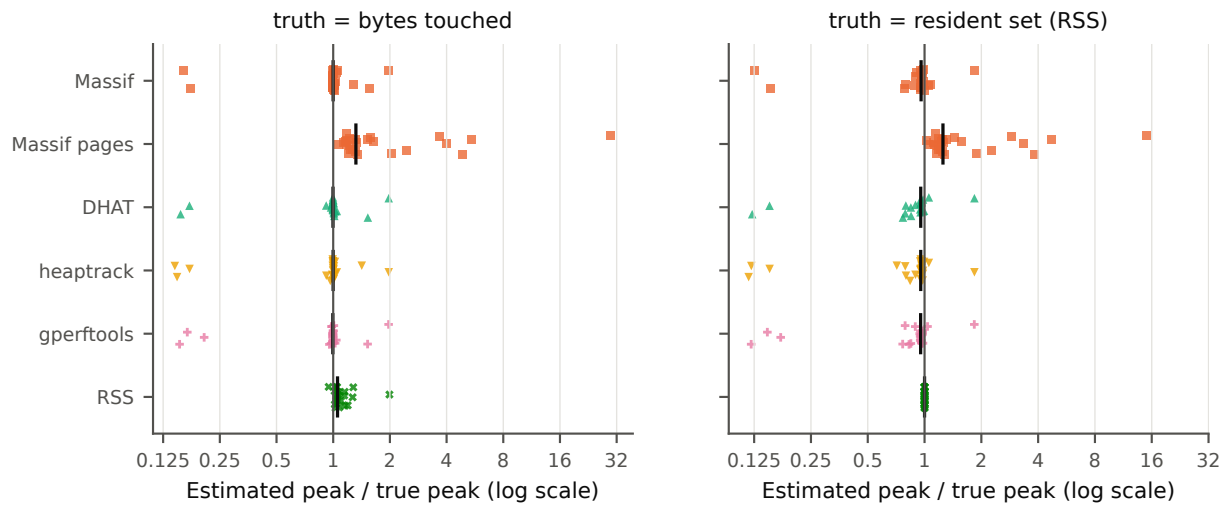


Figure 1: Each method's peak over the true peak, every program at its largest input (one marker per program, the black bar is the median). Left: against bytes touched. Right: against the RSS.

4.1 Accuracy of the peak

The heap profilers report the right peak for most of these programs and fail badly on the rest (Figure 1). Specifically, Massif's peak is within 5% of the true peak on 20 of the 25 programs, DHAT and gperftools on 20 and heaptrack on 19, because these programs touch nearly all the heap they allocate. However, the heap profilers read 0.14 to 0.16 times the peak of GCC's cc1, whose garbage collector maps its own pages, and 0.17 times that of llama2.c, whose weights are mapped from the file and never pass through the heap; they read 1.97 times the peak of darknet and 1.42 to 1.55 times that of miniVite, which allocate memory they never touch. Massif with `-pages-as-heap` overestimates every program (1.07 to 29.9 times), and the RSS is 0.95 to 1.99 times the bytes touched (median 1.05). A peak estimate is therefore right or wrong by program, not by a small error spread over all programs.

4.2 Shape over time

The footprint of half of the programs varies over the run. The mean over the run is below 70% of the peak for 11 of the 25 programs, and it is 0.36 to 0.40 times the peak for the four GAP kernels, so a scheduler that holds the peak for such a job reserves memory that the job leaves unused for most of its run. Figure 2 shows Python's footprint, which rises and falls seven times. Massif's curve follows its phases on its instruction clock but sits above it, because Massif counts requested heap bytes and Python allocates memory it never touches (1.29 times the true peak), and DHAT records only the peak, its time and the heap at exit.

The LLM engines differ from the rest of the corpus in where their memory is (Figure 3). Both map the model's weights from the file and read all of them for the first token, so the true footprint reaches 90% of its peak within the first 1 to 2% of a llama2.c run and stays there, and llama.cpp reaches it at 12% of a 128-token run. The heap profilers see only the heap, 0.08 to 0.21 of llama2.c's peak, and the KV cache and compute buffers of llama.cpp.

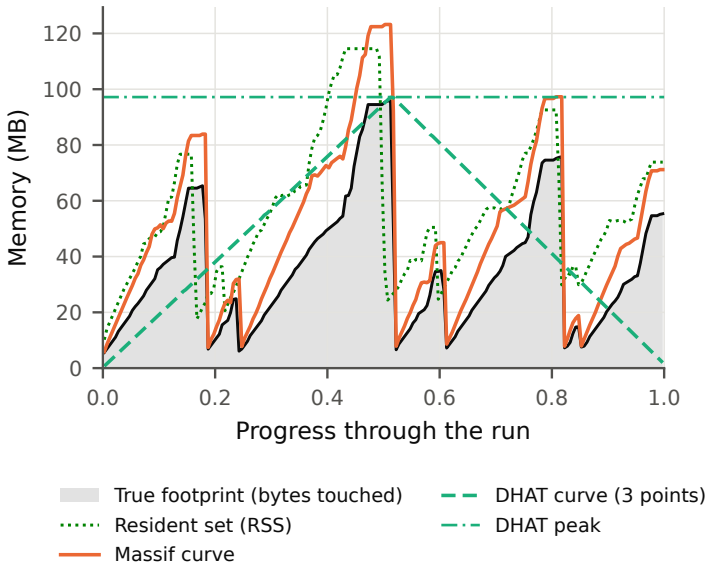


Figure 2: Python (largest input): the true footprint over the run, the RSS, and the curves and peaks the heap profilers report, each on its own method's clock rescaled to progress.

4.3 Cost

The exact footprint is expensive to obtain (Figure 4). At the largest inputs the full instrumentation trace costs 226.8 times the native run at the median (41.2 to 772.5 times), against 1.4 times for heaptrack, 1.2 times for gperftools, 5.7 times for Massif (6.4 times with its clock in milliseconds) and 23.8 times for DHAT. On llama.cpp, a sub-second run with four threads, the full trace costs 469 to 604 times the native run.

5 Results

We first ask how much an exact footprint is worth and where the gain comes from, then how much of it each existing source recovers and why each falls short. Table 2 gives every oracle at offered load 0.8 with calibrated 90% margins, and Figure 5 gives the response time across loads.

5.1 What an exact footprint is worth

Knowing a job's footprint curve, rather than its peak, is where most of the gain lies. At load 0.8 the exact curve gives a mean response time of 84 minutes, against 448 minutes for the exact peak and 2,208 minutes for the user's request, under the bytes-touched truth, which makes the curve 26.4× better than the request and 5.3× better than the peak, and the peak 4.9× better than the request. The RSS truth gives the same picture (99, 423 and 2,183 minutes). The curve lets the scheduler keep 81% of all memory resident with jobs, against 58% for the exact peak and 27% for the request, because jobs whose peaks fall at different times share the memory that a peak would hold idle. Coarsening the exact curve to the 20 widened windows used for every measured curve raises the response to 173 minutes, so most of the curve's advantage over the peak survives the resolution and timing slack a measured curve needs. The gap widens with load (Figure 5): at load 0.4 every oracle that does not over-reserve keeps up with arrivals, and from load 0.6 the peak and

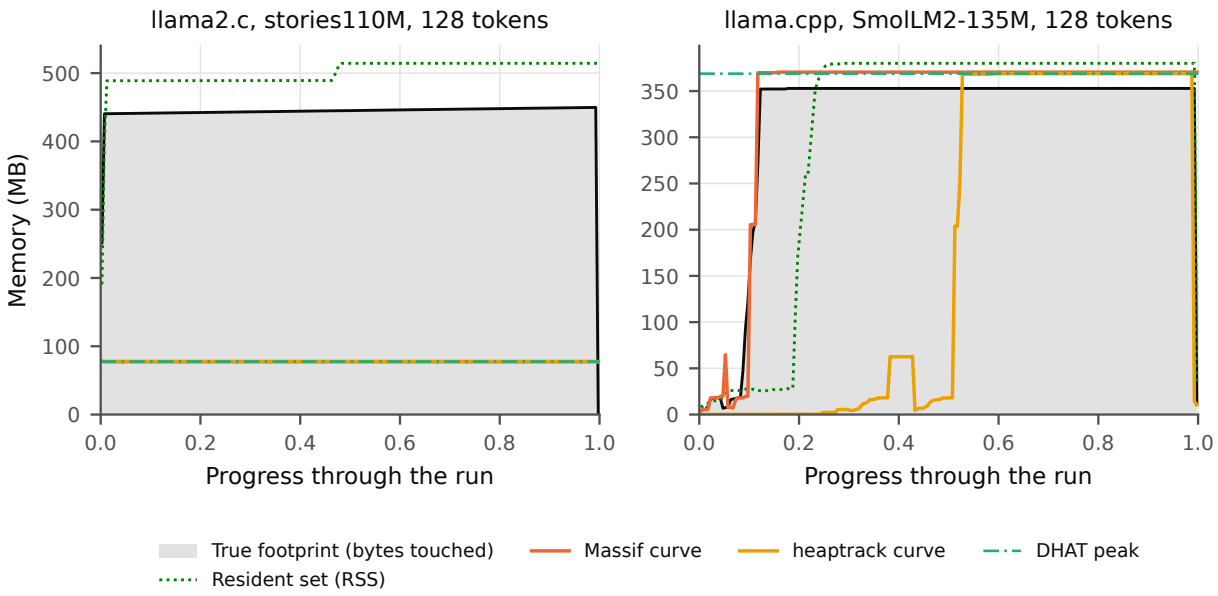


Figure 3: The two LLM engines at their largest inputs: the true footprint over the run, the RSS, and the curves and peaks the heap profilers report.

the request saturate memory while the curve does not.

5.2 Existing footprint estimators

No existing estimator comes close to the exact peak, let alone the exact curve. With calibrated 90% margins, the RSS peak of an earlier run gives 985 minutes and the DHAT peak 1,011, 2.2 times the exact peak, and the ML peak 1,991, little better than the request it scales, because its 0.95-quantile prediction of u covers only 76% of jobs from unseen users. Massif's curve gives 2,399 minutes, worse than the request, because the headroom that covers 90% of the programs is 52% of its peak (Table 3). Three kinds of program set that headroom: Massif reads GCC's cc1 at 0.16 times its peak and llama2.c at 0.17, and on the four GAP kernels its clock of instructions drifts from the truth's clock enough to need 12 to 53% headroom. With Massif's clock in milliseconds under Valgrind, which includes Valgrind's start-up and runs at a different pace from the native program, the headroom rises to 100% and the response time to 4,268 minutes. For gperftools and heaptrack a third program falls above the 90th percentile, Python, whose heap they read at 0.21 and 0.14 times its peak, so every job reserves three to four times its estimate and the response time rises to 10,476 and 11,778 minutes, about five times the request's. The time axis of a curve is thus as important as its values, and an estimator that sees only the heap is wrong by large factors on exactly the programs that set a site's margins.

No margin policy rescues an estimator that is badly wrong for some programs (Figure 6). A small margin lets those programs' jobs be killed, and a margin large enough to cover them over-reserves every other job. With calibrated 100% margins every heap profiler must cover GCC's cc1, so each job reserves about 6.5 times its estimate and the response time rises to 15,000 to 17,000 minutes. With a fixed 10% margin, which accepts the kills, Massif gives 991 minutes with 1,714 kills per stream, and heaptrack and gperftools 1,154 and 1,158 minutes.

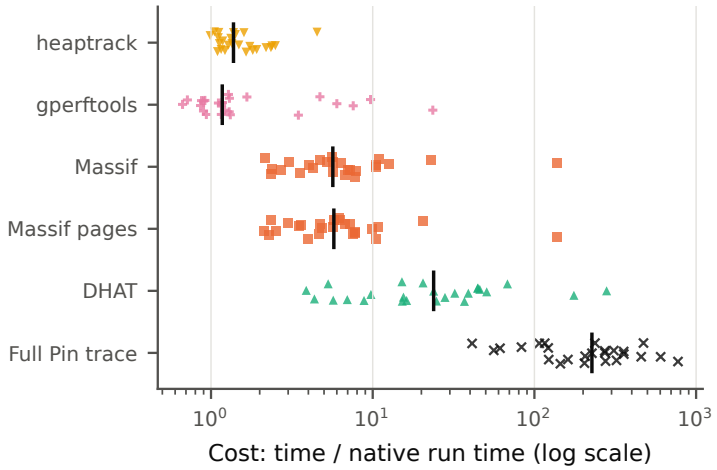


Figure 4: Each method's time over the native run, every program at its largest input. The black bar is the median.

5.3 Published provisioning policies

The published policies recover part of the gain, each for a reason that a scheduler cannot rely on. *Dynamic provisioning* gives 457 minutes under the bytes-touched truth, close to the exact peak (448), and 395 minutes under the RSS truth, with 307 and 265 restarts per stream. It comes this close because each resize is the largest use of the coming five minutes, read from the truth, so between resizes it is an exact curve at a five-minute resolution with no margin. Its look-back variant, which knows only the past as a running system does, gives 504 and 493 minutes, behind the exact peak, with 444 and 478 restarts and 25 and 27 jobs falling back to their request per stream. Reactive resizing, which also looks back but reserves its resizes when the job starts, gives 580 and 612 minutes. A policy that resizes a running job thus reaches the exact peak only with a view of the job's future, and pays for growth it does not foresee with restarts.

Pond gives 2,519 minutes, worse than the request (2,208), with no kills. It holds the whole request, so it cannot pack more jobs than the request does, and the share it places in the pool must fit there even when the node's local memory is free. Its model predicts 8% of the request as untouched against 38% in truth, and still overpredicts for 11% of the jobs, because they come from users the model has not seen. Pond saves local DRAM, not the total memory that the job streams contend for.

KS+ gives 1,262 minutes under the bytes-touched truth and 844 under the RSS truth, with about 8,200 and 8,500 OOM kills per stream. Its first prediction of the largest input from the two smaller ones is 0.86 to 1.18 times the true RSS peak for 19 of the 24 programs it can predict. Two programs cause nearly all the kills: the smaller inputs of atax and NPB MG barely rise above the runtime's baseline, so the regressions predict almost nothing for the largest input, and their jobs climb from the 100 MB floor in 20% steps, 36 to 38 times for atax and 20 to 23 times for MG. GCC's cc1 never fails but reserves 8.6 to 10.7 times its peak. A model learned from a job's earlier executions with other inputs is thus only as good as those inputs are close to the job's.

5.4 LLM inference jobs

On a stream of LLM jobs alone, whose memory is mostly read-only model weights, every heap profiler fails (Figure 7, Table 4). They read llama2.c at 0.08 to 0.21 of its peak, so the margin

Table 2: Every oracle at offered load 0.8, calibrated 90% margins (10% for the request, ML and reactive oracles; none for Pond, dynamic provisioning and KS+, whose Kills column counts failed resizes and retries), mean of three seeds. Resp. is the mean response time, BSLD the mean bounded slowdown, Kills the OOM kills per stream of 3,000 jobs, Res. and Used the memory reserved and resident as a share of all memory.

	Truth = bytes touched					Truth = RSS				
Oracle	Resp. (min)	BSLD	Kills	Res. %	Used %	Resp. (min)	BSLD	Kills	Res. %	Used %
Exact curve	84	2.4	0	81	81	99	3.0	0	80	80
Exact, 20 windows	173	6.3	0	80	72	160	5.7	0	80	73
Exact peak	448	18.3	0	78	58	423	17.2	0	77	61
Dynamic provisioning (5 min)	457	18.9	307	74	63	395	16.1	265	75	64
Dynamic provisioning, look-back	504	21.1	444	73	61	493	20.5	478	72	61
Reactive resizing (5 min)	580	24.2	19	71	54	612	25.6	21	70	54
RSS peak (earlier run)	985	41.8	474	69	44	423	17.2	0	77	61
DHAT peak	1011	42.9	1323	70	45	1302	55.6	1310	65	41
KS+ (segments from other inputs)	1262	54.3	8189	59	38	844	35.7	8534	67	48
ML peak (quantile GBM)	1991	86.0	115	61	29	1992	86.1	115	61	31
User request	2208	95.6	0	57	27	2183	94.4	0	57	29
Massif curve	2399	103.8	1107	53	26	2099	90.6	1184	56	30
Pond (untouched in pool)	2519	109.0	0	53	26	2472	106.9	0	53	27
DHAT curve (3 points)	3512	152.9	1228	51	23	3008	130.7	1157	52	25
DHAT hold-then-decay	4161	180.9	1144	48	20	3943	171.4	1152	47	21
Massif curve, ms clock	4268	185.9	1056	45	17	2684	116.3	1190	51	26
Massif pages-as-heap	4670	203.7	668	40	15	4472	194.4	692	42	16
Massif pages-as-heap, ms clock	6619	290.0	465	35	10	5669	247.5	697	37	12
gperftools peak	10476	458.4	1644	33	7	11181	490.8	1644	33	7
heaptrack curve	11778	516.0	1285	31	6	12610	552.6	1439	31	6

calibrated on the other configurations cannot cover llama2.c's jobs: with calibrated margins Massif, DHAT, heaptrack and gperftools give 8,747 to 9,344 minutes, four times the request (2,067). With a fixed 10% margin, llama2.c's jobs are killed 9,050 to 15,661 times per stream until the requeue factor has raised their reservation enough. The best existing source is the RSS peak of an earlier run, 598 minutes under the bytes-touched truth, because the weights the RSS counts are the footprint, against 180 minutes for the exact curve and 307 for the exact peak. Dynamic provisioning gives 386 minutes by reading the coming five minutes, and its look-back variant 528.

5.5 A real trace

On the Lisa trace with each job's measured memory (Figure 8, Table 5), knowing a job's memory matters once memory is scarce. At Lisa's own 103 GB and at 64 GB per node every policy completes the same 9.4 to 9.6 jobs per hour, because most jobs take whole 16-core nodes and use a few GB, so the cores are the bottleneck. From 32 GB per node memory is contended, and the exact curve completes 8.6 jobs per hour against 6.6 for the exact peak (30% more, with a 40% shorter response time), and 18 to 31% more from 16 down to 4 GB. The peak and the request give the same results, because a whole-node job blocks its node whatever memory it reserves. Dynamic provisioning restarts 775 to 2,129 jobs per stream at 32 GB per node and below and gains less than reactive resizing. On the hybrid stream at 32 GB per node (Table 6), the exact curve gives 10,481 minutes, reactive resizing 11,740, the two dynamic-provisioning variants 13,451 and 13,722, the exact peak 15,563, Pond 19,710, KS+ 19,920 and the heap profilers 21,440 to 59,035 under the bytes-touched truth. On the trace's mix of job sizes, runtimes and node counts, the exact curve therefore still leads, the resizing policies still fall between the curve and the peak, and the heap profilers still fall behind the request, although the request now ties the exact peak because most jobs take whole nodes.

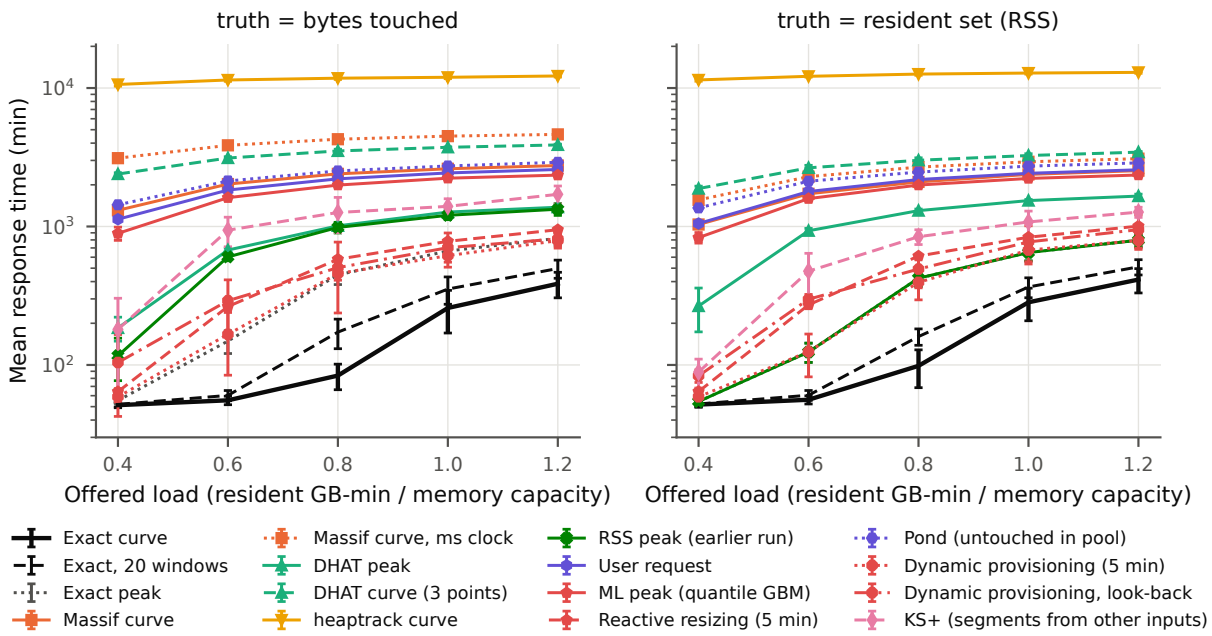


Figure 5: Mean response time against offered load, calibrated 90% margins (95% confidence intervals over three seeds, log scale). Left: the truth is bytes touched. Right: the truth is the RSS.

5.6 Memory needed for the same service

The response-time gains translate into memory a site would have to buy (Figure 9). At a fixed arrival rate, the exact curve meets the target with 5.0 TB in all. Under the bytes-touched truth the exact peak needs 6.0 TB (20% more), reactive resizing and dynamic provisioning 7.0 TB, DHAT, an earlier run's RSS, the ML peak and KS+ 9.0 TB, Massif's curve and the request 11.0 TB (120% more), the DHAT curve 13.0 TB, and Massif on its millisecond clock and Pond 17.0 TB. gperftools and heaptrack do not meet the target even at 512 GB per node.

5.7 What each source's cost buys

The cost of a footprint and its value are far apart for every existing source (Figure 10). Under the bytes-touched truth the full trace costs 227 times the native run and gives a 26.4× gain over the request as a curve, or 4.9× as a peak. heaptrack and gperftools cost under 1.5 times the run and give 1.9×, DHAT costs 24 times and gives 2.2×, Massif's curve costs 5.7 times and gives 2.2× (1.3× with its millisecond clock), and an earlier run's RSS costs nothing extra and gives 2.2×. A source that is cheap gives a tenth of the exact curve's gain, and the source that gives the exact curve costs two orders of magnitude more than the run.

6 DHAT curves

DHAT records the heap at its global peak, the time of that peak, and the heap at exit, but no time series [20]. A scheduler can use those three points to build a *DHAT curve*, which rises linearly from zero to the peak at its recorded time and falls to the exit value, or a *hold-then-decay* curve, which holds the peak until its recorded time and falls afterwards.

Table 3: The margin each oracle needs to stay above the truth, as a share of its own peak (%), over the 25 programs at their largest inputs.

	Truth = bytes touched			Truth = RSS		
Oracle	Median	90th pct.	Max	Median	90th pct.	Max
Massif curve	0	52	534	2	56	689
Massif pages-as-heap	0	28	62	0	30	63
Massif curve, ms clock	0	100	535	20	71	690
Massif pages-as-heap, ms clock	0	65	78	0	52	76
heaptrack curve	1	326	593	2	376	747
DHAT peak	0	6	546	2	24	706
DHAT curve (3 points)	36	90	580	53	99	739
DHAT hold-then-decay	21	90	580	39	99	739
gperftools peak	0	233	554	2	290	716
RSS peak (earlier run)	0	1	7	0	0	0

Both DHAT curves do worse than the DHAT peak under every margin policy and both truths, except with calibrated 100% margins, where all three reserve for GCC's cc1 (Figure 11). At load 0.8 with calibrated 90% margins, the DHAT curve gives 3,512 minutes and the hold-then-decay curve 4,161, against 1,011 for the DHAT peak, under the bytes-touched truth. The curves fall below the truth wherever a program has more than one phase, and the median program needs 36% headroom on the DHAT curve. The information that makes a curve useful to a scheduler is the timing of every phase, and DHAT records the timing of one.

7 Requirements for a footprint estimator

The results above determine what a source of footprint knowledge must provide for a scheduler to recover the gain of an exact footprint.

- **A curve, not a peak.** The exact curve is 5.3 times better than the exact peak at load 0.8 and needs 20% less memory for the same service, so an estimator must report the footprint over the run.
- **A time axis that matches the run.** A curve on a clock other than the job's own (Massif's milliseconds under Valgrind, or its instructions on the GAP kernels) needs headroom that erases its advantage, and DHAT's single recorded time does not make a usable curve.
- **All of the memory the program touches, and only that.** The heap profilers miss memory a program maps itself (cc1's collector, the LLM engines' weights) and count memory it allocates and never touches (darknet, miniVite), and those programs set every site-wide margin.
- **A small margin that holds across programs.** An estimate that is exact for most programs and wrong by a large factor for a few forces a margin that over-reserves every job.
- **Known before the job starts.** Dynamic provisioning reaches the exact peak only by reading the job's future, and without it falls behind the peak and restarts hundreds of jobs per stream; Pond and KS+ need history that a new user or a new input does not have.

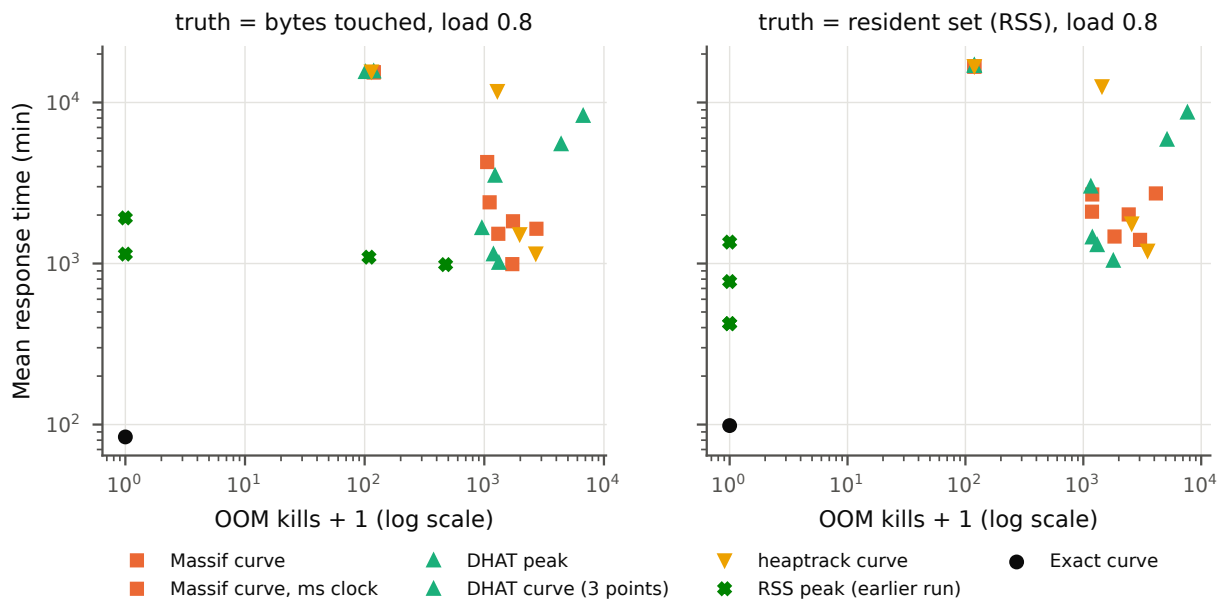


Figure 6: Mean response time against OOM kills at load 0.8, one marker per margin policy (10%, 30%, calibrated 90% and 100%). No margin gives an existing estimator both few kills and a short response time.

- **A cost close to the run's.** The full trace gives the exact curve at 227 times the native run, which no site can pay for every job.

8 Limitations

The results rest on a model of a cluster and of its jobs, and each simplification limits what they show.

The published baselines. Our Pond follows its paper, because its artifact holds none of its policies, and it leaves out the model that places latency-insensitive virtual machines entirely in the pool, which needs hardware counters our jobs do not have. Our dynamic provisioning fixes a job's runtime stretch at admission and charges nothing per resize, both in its favour. KS+ learns from two smaller inputs per program, where its authors had hundreds of executions per task.

Scaled footprints. The corpus programs peak at 15 to 450 MB, and the simulated jobs at tens to hundreds of GB. Scaling a measured curve to a job's size keeps its shape and every estimator's ratio to the truth, but it assumes the ratio does not change with the input, so we use the largest input of every program.

The job streams. The main stream is synthetic, with memory the contended resource by design, and the used-to-requested ratio comes from the LANL CM5 log of 1994 to 1996. The Lisa trace's jobs are those that had their nodes to themselves, its memory is the nodes' active memory, and it records no memory request.

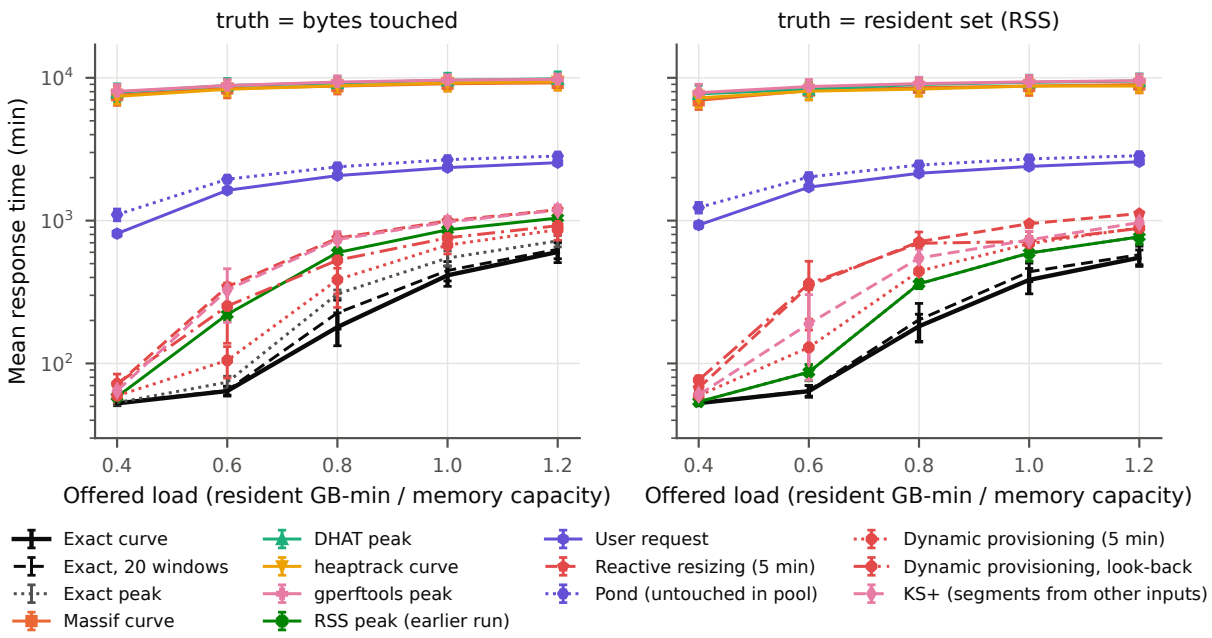


Figure 7: A stream of LLM inference jobs alone (llama2.c with three models, llama.cpp with two lengths): mean response time against offered load, calibrated 90% margins, each configuration's margin calibrated on the other four.

Scheduler and machine. The slowdown is linear in the remote share of the footprint with one β for every job, the pool has no bandwidth contention, and only FCFS with EASY backfilling was tested. Appendix A shows that the orderings hold for β from 0.1 to 2 and pools of 0.5 to 4 TB, and that the update interval of the resizing policies changes their results the most.

Costs. The costs were measured with up to 15 runs sharing the node, and Pin's fixed start-up cost weighs heavily on runs of a few seconds.

9 Conclusion

An exact footprint is worth most to a scheduler on disaggregated memory as a curve over the job's run, and the gain comes from letting jobs with peaks at different times share memory: on our corpus and streams, the exact curve gives a 26.4× shorter mean response time than the user's request and needs 5.0 TB where the request needs 11.0 TB, and on the Lisa trace it completes 18 to 31% more jobs than the exact peak once memory is scarce. None of the existing sources recovers that gain. The heap profilers are wrong by large factors on the programs that map their memory or never touch what they allocate, and no margin covers those programs without over-reserving the rest. Of three published policies, dynamic provisioning comes close to the exact peak only by reading each job's coming use, Pond saves local DRAM but not total memory, and KS+ cannot extrapolate from inputs far from the job's. The exact curve itself costs 227 times the run. A footprint estimator for a scheduler must therefore report a curve on the run's own time axis, count all the memory a program touches and nothing it does not, hold with a small margin across programs, be known before the job starts, and cost close to the run. We present a tool that targets these requirements in companion work [14].

Table 4: The LLM stream at offered load 0.8, calibrated 90% margins; columns as in Table 2.

	Truth = bytes touched					Truth = RSS				
Oracle	Resp. (min)	BSLD	Kills	Res. %	Used %	Resp. (min)	BSLD	Kills	Res. %	Used %
Exact curve	180	6.6	0	76	76	182	6.7	0	74	74
Exact, 20 windows	226	8.6	0	75	72	202	7.5	0	76	73
Exact peak	307	12.1	0	75	69	361	14.4	0	76	66
Dynamic provisioning (5 min)	386	15.8	140	72	65	443	18.3	200	71	63
Dynamic provisioning, look-back	528	22.2	260	68	62	694	29.3	356	62	56
RSS peak (earlier run)	598	24.9	0	69	57	361	14.4	0	76	66
KS+ (segments from other inputs)	737	31.1	1542	64	49	547	22.8	797	69	56
Reactive resizing (5 min)	753	31.9	10	65	54	712	29.9	9	66	54
ML peak (quantile GBM)	1836	79.1	115	61	36	1904	82.1	115	61	34
User request	2067	89.3	0	58	33	2146	92.7	0	57	31
Pond (untouched in pool)	2379	102.8	0	52	30	2450	105.9	0	53	28
Massif pages-as-heap	2650	114.7	0	47	26	2186	94.5	0	52	30
Massif pages-as-heap, ms clock	2955	127.9	0	45	24	2495	107.9	0	48	26
Massif curve	8747	381.2	1195	32	9	8528	372.3	1195	32	10
heaptrack curve	8794	383.2	1195	32	9	8333	363.2	1195	32	10
Massif curve, ms clock	8869	386.7	1195	32	9	8630	376.5	1195	32	10
DHAT curve (3 points)	9103	397.6	1195	31	9	8776	383.4	1195	32	9
DHAT peak	9198	400.7	1195	31	9	8986	391.7	1195	32	9
gperftools peak	9344	408.0	1195	31	9	9108	397.5	1195	32	9
DHAT hold-then-decay	9561	417.3	1195	31	8	9251	404.3	1195	32	9

Table 5: The measured Lisa stream at offered load 0.8: jobs per hour and mean response time for each local memory per node (mean of three seeds). The load is set against each cluster's memory, so numbers compare across rows, not across columns.

	Jobs per hour						Mean response (min)					
Local GB per node	103	64	32	16	8	4	103	64	32	16	8	4
Exact curve	9.6	9.4	8.6	3.5	3.8	2.0	11,790	10,755	9,735	36,491	36,939	14,073
Exact, 20 windows	9.6	9.4	7.5	3.2	3.5	1.9	11,790	10,755	12,264	46,250	46,383	20,655
Exact peak	9.6	9.4	6.6	2.7	2.9	1.7	11,790	10,755	16,080	59,143	60,184	28,962
User request	9.6	9.4	6.6	2.7	2.9	1.7	11,790	10,755	16,080	59,143	60,184	28,962
Reactive resizing (5 min)	9.6	9.4	7.5	3.0	3.4	1.8	11,790	10,755	11,705	46,413	44,847	18,800
Dynamic provisioning (5 min)	9.6	9.4	6.4	3.0	2.8	1.5	11,787	10,755	20,327	59,675	51,699	26,652
Dynamic provisioning, look-back	9.6	9.4	7.3	2.1	3.0	1.7	11,787	10,755	14,802	58,906	56,997	14,948

A Sensitivity to the simulator's assumptions

We varied each of the simulator's assumptions alone, at offered load 0.8. The remote-memory slowdown β changes every oracle's response time, from 58 to 390 minutes for the exact curve between $\beta = 0.1$ and 2, but not their order. The pool decides how much memory is contended: with 4 TB most oracles come within a few minutes of the runtimes, and with 512 GB the differences grow (the exact curve 565 minutes, the exact peak 1,499, the request 4,441). The update interval matters most for the policies that resize a running job: dynamic provisioning gives 157 minutes with an update every minute and 1,330 with one every 30 minutes under the bytes-touched truth, and its look-back variant 182 to 1,657 minutes.

B Reproducing the baselines' own results

We reran each baseline's own evaluation where its data or a public substitute exists. The authors of KS+ publish their simulation and traces, and every reduction their paper reports is reproduced

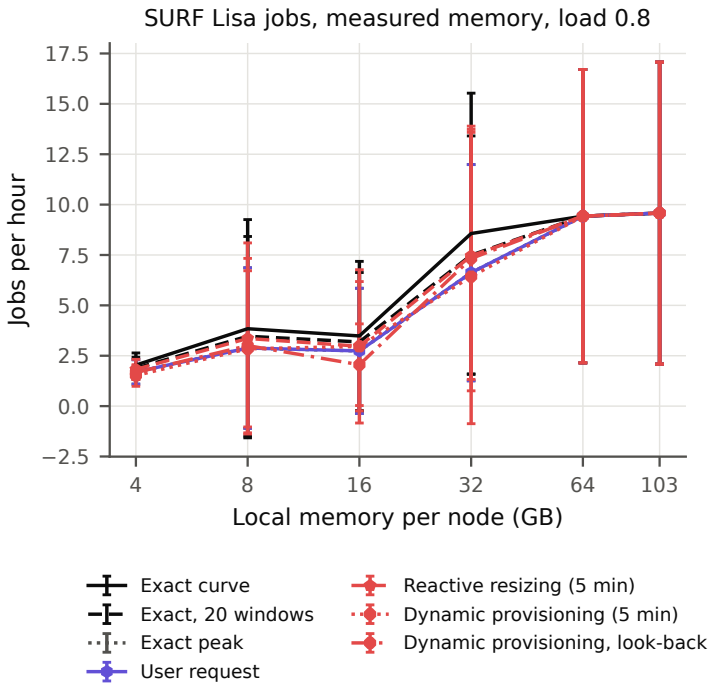


Figure 8: SURF Lisa jobs with their measured memory on a Lisa-shaped cluster, offered load 0.8: jobs per hour against local memory per node (mean and 95% interval over three seeds).

within 2 percentage points. Zacarias et al. publish an artifact with their Slurm simulator, trace generation and a test set of nine simulations with output checksums; built and run as its appendix prescribes, with its R packages taken from a CRAN snapshot of the artifact's date, all nine outputs match the authors' checksums bit for bit. Pond's results rest on Azure's internal telemetry and its artifact holds none of its policies, so only its Figure 3, the DRAM that pooling saves, can be rebuilt, from Azure's public packing trace. Provisioning each server for its own peak reproduces the figure's shape (savings rising with pool size, more slowly beyond 16 sockets) at a sixth of its magnitude, and provisioning every server for the cluster's largest peak reproduces its magnitude without its shape; the public trace lacks the server types and placements Pond used.

References

[1] Emmanuel Amaro et al. Can far memory improve job throughput? In *European Conference on Computer Systems (EuroSys)*, 2020.

[2] Jonathan Bader, Ansgar Lößer, Lauritz Thamsen, Björn Scheuermann, and Odej Kao. KS+: Predicting workflow task memory usage over time. In *IEEE 20th International Conference on e-Science*, 2024.

[3] Robin Boezennec et al. Allocation strategies for disaggregated memory in HPC systems. In *IEEE International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2024.

[4] CXL-ClusterSim: Modeling CXL-based disaggregated memory cluster using gem5 and SST. arXiv:2605.27745, 2026.

Table 6: The hybrid Lisa stream (the trace's jobs with corpus footprint shapes) at 32 GB per node and offered load 0.8, calibrated 90% margins. Fails counts OOM kills, restarts and retries.

	Truth = bytes touched			Truth = RSS		
Oracle	Jobs/h	Resp. (min)	Fails	Jobs/h	Resp. (min)	Fails
Exact curve	6.9	10,481	0	6.8	10,590	0
Exact, 20 windows	6.7	11,585	0	6.7	12,289	0
Reactive resizing (5 min)	6.7	11,740	2	6.5	12,983	4
Dynamic provisioning, look-back	6.6	13,451	251	6.6	13,749	282
Dynamic provisioning (5 min)	6.6	13,722	405	6.2	14,945	515
User request	6.3	15,563	0	6.3	16,012	0
Exact peak	6.3	15,563	0	6.3	16,012	0
Pond (untouched in pool)	6.0	19,710	0	6.0	19,677	0
KS+ (segments from other inputs)	4.8	19,920	6136	3.3	17,157	6411
RSS peak (earlier run)	5.8	20,623	474	6.3	16,012	0
DHAT peak	5.2	21,440	1323	4.9	24,514	1310
ML peak (quantile GBM)	4.1	21,892	212	4.0	21,682	212
Massif curve	3.9	31,021	1092	3.8	30,744	1188
DHAT curve (3 points)	3.1	45,306	1197	3.1	38,652	1149
Massif curve, ms clock	3.2	46,978	1040	3.6	34,698	1190
heaptrack curve	4.1	47,648	1285	3.8	45,731	1440
gperftools peak	3.3	59,035	1644	3.6	57,569	1644

Table 7: Mean response time (minutes) at offered load 0.8 as the remote-memory slowdown β varies (default 0.5), calibrated 90% margins, mean of three seeds.

	Truth = bytes touched					Truth = RSS				
β	0.1	0.25	0.5	1	2	0.1	0.25	0.5	1	2
Exact curve	58	63	84	190	390	58	67	99	201	470
Exact peak	274	340	448	693	1,162	233	302	423	666	1,194
DHAT peak	775	927	1,011	1,360	1,975	1,001	1,122	1,302	1,634	2,379
User request	1,896	2,012	2,208	2,601	3,365	1,846	1,977	2,183	2,627	3,431
Reactive resizing (5 min)	432	500	580	779	1,273	446	497	612	838	1,343
Pond (untouched in pool)	2,188	2,301	2,519	2,903	3,649	2,160	2,271	2,472	2,925	3,748
Dynamic provisioning (5 min)	286	332	457	568	1,021	274	363	395	594	932
Dynamic provisioning, look-back	481	439	504	686	1,121	460	507	493	706	1,064
KS+ (segments from other inputs)	1,031	963	1,262	1,378	1,828	704	741	844	1,215	1,282

[5] CXL-DMSim: A full-system CXL disaggregated memory simulator. arXiv:2411.02282, 2024.

[6] Dror G. Feitelson, Dan Tsafrir, and David Krakov. Parallel workloads archive. `https://www.cs.huji.ac.il/labs/parallel/workload/`. Logs LANL CM5 (LANL-CM5-1994-4.1-cln) and MetaCentrum2 (METACENTRUM-2013-3).

[7] Georgi Gerganov and ggml contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, release b11500, 2026.

[8] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, and Assefaw H. Gebremedhin. miniVite: A graph analytics benchmarking tool for massively parallel systems. In *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems Workshop (PMBS)*, 2018.

[9] gperftools: TCMalloc and heap profiler. `https://github.com/gperftools/gperftools`. Version 2.18.

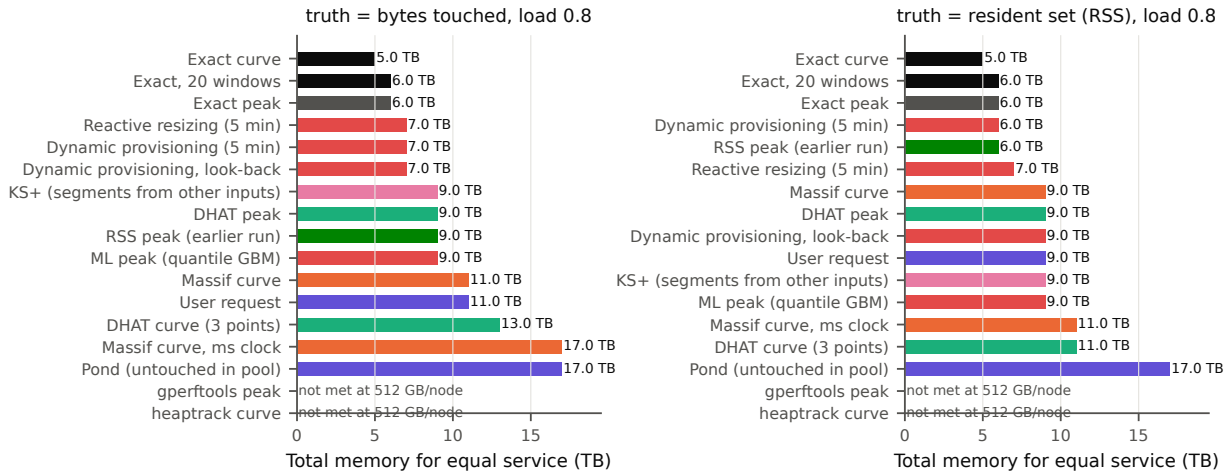


Figure 9: The total memory (32 nodes of local DRAM plus the 1 TB pool) each oracle needs to keep the mean bounded slowdown within 1.5 times that of the exact curve at 128 GB per node, at load 0.8. Local memory per node was swept from 48 to 512 GB.

Table 8: Mean response time (minutes) at offered load 0.8 as the pool varies (default 1 TB).

	Truth = bytes touched				Truth = RSS			
Pool (GB)	512	1024	2048	4096	512	1024	2048	4096
Exact curve	565	84	51	51	671	99	51	51
Exact peak	1,499	448	78	52	1,455	423	65	52
DHAT peak	2,354	1,011	262	65	2,818	1,302	354	78
User request	4,441	2,208	697	261	4,458	2,183	667	222
Reactive resizing (5 min)	1,669	580	94	52	1,708	612	90	52
Pond (untouched in pool)	4,982	2,519	802	258	4,976	2,472	770	221
Dynamic provisioning (5 min)	1,624	457	58	50	1,617	395	57	50
Dynamic provisioning, look-back	1,887	504	56	50	2,134	493	56	50
KS+ (segments from other inputs)	2,005	1,262	437	126	1,669	844	350	113

[10] Andrej Karpathy. llama2.c: Inference Llama 2 in one file of pure C. `https://github.com/karpathy/llama2.c`, 2023.

[11] Huaicheng Li et al. Pond: CXL-based memory pooling systems for cloud platforms. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[12] David A. Lifka. The ANL/IBM SP scheduling system. Job Scheduling Strategies for Parallel Processing (JSSPP), LNCS 949, 1995.

[13] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.



Figure 10: Profiling cost per job (median time over the native run) against the gain in mean response time over the user's request, at load 0.8, each oracle at its best margin policy. The exact curve's cost is the full trace under the bytes-touched truth and an RSS sampler under the RSS truth.

Table 9: Mean response time (minutes) at offered load 0.8 as the update interval of the policies that resize a running job varies (default 5 minutes).

	Truth = bytes touched				Truth = RSS			
Update interval (min)	1	5	15	30	1	5	15	30
Reactive resizing	321	580	1,034	1,402	329	612	1,063	1,421
Dynamic provisioning	157	457	812	1,330	182	395	872	1,232
Dynamic provisioning, look-back	182	504	1,175	1,657	191	493	1,029	1,758

[14] Nafis Mustakin, Joseph Manzano, and Daniel Wong. MemPrint: Constructing program memory footprint estimations using statistical methods from sparsely sampled Pin-based memory traces. Manuscript, 2025.

[15] Ivy Peng et al. On the memory underutilization: Exploring disaggregated memory on HPC systems. In *International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, 2020.

[16] Louis-Noël Pouchet. PolyBench/C 4.2.1: The polyhedral benchmark suite, 2016. `https://sourceforge.net/projects/polybench/`.

[17] Amit Puri et al. DRackSim: Simulator for rack-scale memory disaggregation. In *ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS)*, 2024. arXiv:2305.09977.

[18] Julian Seward and Nicholas Nethercote. *Valgrind Massif: Heap Profiler*, 2015. Available at `http://valgrind.org/docs/manual/ms-manual.html`.

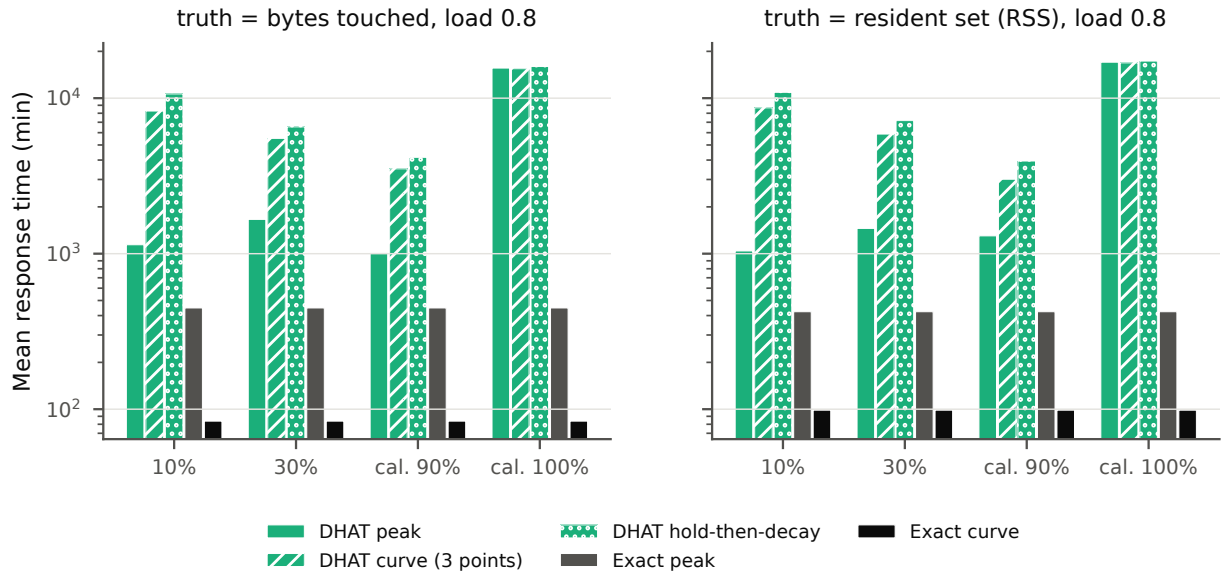


Figure 11: DHAT's peak and the two curves built from its three recorded points, against the exact peak and curve, at load 0.8 under each margin policy (log scale).

Table 10: KS+'s memory wastage reduction against its baselines, as the KS+ paper reports it and as the authors' code gives it on their traces, for 25, 50 and 75% training data.

		Paper			Reproduced		
Workflow	KS+ against	25%	50%	75%	25%	50%	75%
eager	k-Segments Selective	36	39	40	36	40	40
eager	PPM-Improved	54	52	51	54	52	51
sarek	k-Segments Selective	31	28	29	32	28	30
sarek	PPM-Improved	45 (mean)			47	46	46

[19] SURF. Generic and ML workloads in an HPC datacenter: SURF Lisa slurm and prometheus traces. Zenodo, doi:10.5281/zenodo.13625495, CC BY 4.0, 2024.

[20] Valgrind Developers. *DHAT: a dynamic heap analysis tool*. Available at `https://valgrind.org/docs/manual/dh-manual.html`.

[21] Milian Wolff. heaptrack: a heap memory profiler for Linux. `https://github.com/KDE/heaptrack`. Version 1.3.0.

[22] Yiwei Yang et al. CXLMemSim: A pure software simulated CXL.mem. arXiv:2303.06153, 2023.

[23] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Improving HPC system throughput and response time using memory disaggregation. `https://upcommons.upc.edu/`, 2021.

[24] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Memory demands in disaggregated HPC: How accurate do we need to be? In *International Workshop on Performance*

Table 11: DRAM saved by pooling a fixed share of every virtual machine's memory, in percent, against the pool size in CPU sockets, from the public Azure packing trace, with each server provisioned for its own peak or every server for the cluster's largest peak; and Pond's Figure 3 at a 50% share.

Provisioning	Pool share	2	4	8	16	32	64
uniform fleet	10%	3.8	3.9	4.0	4.0	4.1	4.1
uniform fleet	30%	11.5	11.8	11.9	12.1	12.2	12.3
uniform fleet	50%	19.2	19.6	19.9	20.2	20.4	20.6
per server	10%	0.0	0.1	0.2	0.3	0.4	0.4
per server	30%	0.0	0.4	0.7	1.0	1.2	1.3
per server	50%	0.0	0.6	1.2	1.6	1.9	2.2
Pond (paper)	50%					12	13

Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS), 2021.

[25] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Dynamic memory provisioning on disaggregated HPC systems. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC-W)*, 2023.